

CLOSED, PROJECT-BASED CURRICULUM

Derivatives Foundation for ML Research

A 15-notebook syllabus for options, volatility, IV surfaces, and the transition into machine learning

Designed as a finite reset: learn by implementing, pass each notebook by explanation, then return your main effort to ML/DL.

Research anchor

Kelly, Kuznetsov, Malamud, and Xu - Deep Learning from Implied Volatility Surfaces (2026)

Version 1.0 | September 2026

What this curriculum is for

This syllabus is a compact derivatives foundation for a machine-learning researcher working with options and implied-volatility surfaces. It is intentionally narrower than a finance degree and deeper than a collection of disconnected videos. The learning unit is a reproducible notebook, not a lecture watched.

FINISH LINE

When the full chain from real-world stock dynamics to an ML-ready IV tensor feels obvious, stop broad finance study and return your primary effort to statistics, ML, deep learning, and research design.

1
MARKETS AND PRICING LOGIC

Notebooks 1-4: returns, arbitrage, trees, Black-Scholes

2
SENSITIVITIES AND REPLICATION

Notebooks 5-7: Greeks, delta hedging, option P&L;

3
IMPLIED VOLATILITY AND MEASURES

Notebooks 8-11: IV, surfaces, P vs Q, Monte Carlo

4
SURFACES TO MACHINE LEARNING

Notebooks 12-15: Heston, cleaning, tensors, research

Operating rules

- Write the core numerical logic yourself before using specialized option libraries.
- Every notebook must contain a question, an implementation, an experiment, a result, and a written interpretation.
- Use fixed random seeds and save parameters so every figure can be reproduced.
- Separate market conventions and data-cleaning assumptions from model code.
- Use chronological splits whenever prediction enters the notebook.
- Do not mark a notebook complete because it runs. Mark it complete only when you can explain its result without notes.

What is deliberately outside scope

CAPM, corporate finance, fixed-income term-structure models, stochastic discount factors in depth, exotic derivatives, advanced Girsanov proofs, local-volatility calibration, SABR, HJM, and production Heston calibration. These can be learned later if a concrete research question requires them.

Definition of done for every notebook

Notebook structure

- Question and financial intuition in your own words
- Assumptions and units table
- Implementation from first principles
- Sanity checks against identities or closed forms
- Experiments that change one variable at a time
- Plots with labeled axes and units
- Written interpretation and limitations

Reproducibility package

- Pinned environment or requirements file
- Fixed random seed where relevant
- Reusable functions separated from analysis cells
- No hidden notebook state
- Clean run from top to bottom
- Small exported results table
- One paragraph: what would fail on real data?

Recommended cadence

Window	Notebooks	Focus	Expected output
Days 1-4	1-4	Core pricing chain	GBM, parity, tree, BS pricer
Days 5-7	5-7	Risk and replication	Greeks, hedge errors, P&L attribution
Days 8-11	8-11	Volatility and measures	IV solver, surface, P/Q, MC
Days 12-15	12-14	Surface generation and data	Heston surface, checks, tensor
Days 16-18	15 + review	Research integration	Baseline study and oral defense

TIME RULE

One notebook every 1-2 focused days is a target, not a deadline. If a pass criterion fails, repair the conceptual gap before advancing. If it passes, move on without adding optional theory.

Contents

- | | |
|---|---|
| 01 Returns, volatility, and GBM | 02 No-arbitrage, forwards, and put-call parity |
| 03 Binomial option pricing | 04 Black-Scholes from scratch |
| 05 Greeks | 06 Delta hedging |
| 07 Option P&L; decomposition | 08 Implied volatility |
| 09 Volatility smile and IV surface | 10 Physical P versus risk-neutral Q |
| 11 Monte Carlo option pricing | 12 Heston simulation and an emergent smile |
| 13 Option-surface arbitrage sanity checks | 14 From option chain to ML tensor |
| 15 Tiny end-to-end volatility research project | |

Returns, Volatility, and Geometric Brownian Motion

PURPOSE

Build the distinction between one historical time series and many simulated paths, and learn what drift and volatility parameters mean before using them in an option model.

Core concepts

- Simple versus log returns
- Daily versus annualized volatility
- GBM and the role of Brownian increments
- Parameter estimation from prices
- Time-series observations versus Monte Carlo paths

Implementation checklist

- Load or create a clean price series
- Compute both return definitions
- Estimate daily and annualized mu and sigma
- Implement exact GBM simulation
- Support 1, 100, and 10,000 paths
- Re-estimate parameters from simulated data

Key equations

```
simple_return[t] = (S[t] - S[t-1]) / S[t-1]
log_return[t] = log(S[t] / S[t-1])
dS = mu*S*dt + sigma*S*dW
S[t+dt] = S[t]*exp((mu - 0.5*sigma^2)*dt + sigma*sqrt(dt)*Z)
```

Required experiments

- [] Compare histograms of simple and log returns
- [] Plot one path beside a historical series without implying equivalence
- [] Plot terminal-price distributions for increasing path counts
- [] Repeat estimation across sample lengths
- [] Demonstrate that drift estimates remain noisy while volatility stabilizes

Evidence to save

- Price and return plots
- Parameter-estimation table
- Terminal-price histogram
- Short note on estimation uncertainty

Common failure modes

- Mixing daily and annual units
- Using arithmetic mean as continuous drift without adjustment
- Treating simulated paths as independent historical assets
- Forgetting the $-0.5 \sigma^2$ correction

PASS CRITERION

Explain why $dW = \sqrt{dt} Z$, why 100 simulated paths are not 100 observations from one time series, and why drift is harder to estimate than volatility.

No-Arbitrage, Forwards, and Put-Call Parity

PURPOSE

Learn pricing as a consistency problem before treating Black-Scholes as a formula. Build and diagnose portfolios that must have the same terminal payoff.

Core concepts

- Law of one price
- Discounting and carry
- Forward price
- European call-put equivalence
- Intrinsic and lower bounds

Implementation checklist

- Write parity residual functions
- Generate internally consistent synthetic quotes
- Inject controlled violations
- Detect violations with transaction-cost tolerances
- Construct the offsetting long-short portfolio
- Report initial and terminal cash flows

Key equations

$$F_0 = S_0 \exp(rT)$$

$$C - P = S_0 - K \exp(-rT)$$

$$C + K \exp(-rT) = P + S_0$$

$$C \geq \max(S_0 - K \exp(-rT), 0)$$

Required experiments

- [] Vary rates, strike, and maturity
- [] Show a false signal when bid-ask spreads are ignored
- [] Test both underpriced-call and overpriced-call cases
- [] Verify the terminal payoff state by state

Evidence to save

- Parity checker
- Arbitrage cash-flow table
- Payoff diagram
- Tolerance policy

Common failure modes

- Comparing mid prices with executable trades
- Using the wrong discounting sign
- Mixing European and American options
- Calling a small noisy residual a free arbitrage

PASS CRITERION

Given stock, bond, call, and put prices, determine whether an arbitrage exists and describe the exact trades without invoking Black-Scholes.

Binomial Option Pricing

PURPOSE

Make replication and risk-neutral valuation concrete in a finite-state model. This notebook supplies the conceptual bridge to continuous-time option pricing.

Core concepts

- One-period replication
- Multi-period backward induction
- Risk-neutral probability
- Self-financing portfolio
- Convergence toward Black-Scholes

Implementation checklist

- Build a one-step stock and option tree
- Solve Delta and the bond holding
- Generalize to N steps
- Price European calls and puts
- Implement backward induction
- Record price and runtime by N

Key equations

```
u = exp(sigma*sqrt(dt))
d = 1/u
q = (exp(r*dt) - d) / (u - d)
V = exp(-r*dt) * (q*V_up + (1-q)*V_down)
Delta = (V_up - V_down) / (S_up - S_down)
```

Required experiments

- [] Run N = 10, 50, 100, and 500
- [] Confirm q lies in [0,1] under no-arbitrage conditions
- [] Compare replicated and discounted-expectation values
- [] Later compare convergence against Black-Scholes

Evidence to save

- Tree visual for small N
- Replicating-portfolio table
- Convergence plot
- Unit tests for terminal payoffs

Common failure modes

- Using physical probabilities in the pricing recursion
- Off-by-one maturity errors
- Forgetting discounting
- Building full trees when recombining arrays suffice

PASS CRITERION

Explain why q is a pricing probability and not a forecast of the real probability that the stock rises.

Black-Scholes from Scratch

PURPOSE

Implement the canonical European option pricer, connect each input to economic intuition, and verify it against parity and the binomial model.

Core concepts

- European calls and puts
- Moneyness and time value
- Risk-free discounting
- Constant volatility
- Closed-form pricing as a replication result

Implementation checklist

- Implement normal CDF usage
- Implement call and put prices
- Handle $T \rightarrow 0$ explicitly
- Vectorize across strikes or spots
- Verify put-call parity
- Compare with binomial prices

Key equations

```
d1 = [log(S0/K) + (r + 0.5*sigma^2)*T] / (sigma*sqrt(T))
d2 = d1 - sigma*sqrt(T)
Call = S0*N(d1) - K*exp(-r*T)*N(d2)
Put = K*exp(-r*T)*N(-d2) - S0*N(-d1)
```

Required experiments

- [] Plot price versus S , σ , T , and K
- [] Change one input at a time
- [] Check deep ITM and OTM limits
- [] Compare binomial convergence for several strikes
- [] Test monotonicity and bounds

Evidence to save

- Reusable BS functions
- Sensitivity plots
- Binomial comparison table
- Boundary-case tests

Common failure modes

- Using calendar days instead of year fractions
- Passing volatility as 20 instead of 0.20
- Ignoring dividends in market-data applications
- Confusing model price with observed market price

PASS CRITERION

Define every input, state the central assumptions, and predict the direction of a call or put price change before running the code.

Greeks: Analytic and Numerical

PURPOSE

Turn option vocabulary into measurable local sensitivities and verify analytic formulas independently with finite differences.

Core concepts

- Delta, gamma, vega, theta, and rho
- First- versus second-order sensitivity
- ATM, ITM, and OTM behavior
- Finite-difference validation
- Units and sign conventions

Implementation checklist

- Implement analytic Greeks
- Implement central differences
- Compare errors across step sizes
- Support calls and puts
- Document per-1.0 versus per-1% vega
- Document annual versus daily theta

Key equations

$\Delta \approx [C(S+h) - C(S-h)] / (2h)$
 $\Gamma \approx [C(S+h) - 2C(S) + C(S-h)] / h^2$
 $\text{Vega} \approx [C(\sigma+h) - C(\sigma-h)] / (2h)$
 $\Theta = \partial C / \partial t$ (state your time convention)

Required experiments

- [] Plot each Greek versus spot
- [] Overlay several maturities
- [] Inspect behavior near expiry
- [] Sweep finite-difference h
- [] Explain ATM concentration of gamma and vega

Evidence to save

- Greek functions
- Analytic-numeric error table
- Four labeled Greek plots
- Units and conventions box

Common failure modes

- Mismatched theta signs
- Vega scaling errors
- Choosing h too large or too small
- Interpreting a local sensitivity as an exact large-move P&L;

PASS CRITERION

Explain why gamma and vega tend to matter around ATM options, how maturity changes them, and why a numerical Greek can be unstable.

Dynamic Delta Hedging

PURPOSE

Experience Black-Scholes as dynamic replication. Measure how discrete rebalancing, volatility misspecification, and path dependence create hedging error.

Core concepts

- Self-financing hedge
- Option liability versus hedge portfolio
- Discrete rebalancing
- Cash-account accrual
- Model error versus discretization error

Implementation checklist

- Simulate stock paths
- Sell or replicate one European call
- Initialize stock and cash holdings
- Rebalance delta through time
- Accrue cash consistently
- Compute terminal hedge error

Key equations

```
Delta[t] = partial C / partial S
cash[t+] = (portfolio[t] - Delta[t]*S[t]) * exp(r*dt)
hedging_error = hedge_portfolio[T] - option_payoff[T]
```

Required experiments

- [] Use 1, 5, 20, 50, 252, and 1,000 rebalances
- [] Plot the hedge-error distribution
- [] Set true sigma = 20% and hedge sigma = 15%
- [] Repeat across paths with identical parameters
- [] Separate pricing P&L; from hedge error

Evidence to save

- One-path hedge ledger
- Error distribution plots
- Frequency-versus-error summary
- Wrong-volatility comparison

Common failure modes

- Wrong sign for a short option
- Recomputing option value but not financing the hedge
- Look-ahead rebalancing
- Comparing simulations with different seeds when isolating one change

PASS CRITERION

Explain why replication is exact only in the model limit, why more frequent hedging reduces but does not erase all real-world risk, and how incorrect volatility changes outcomes.

Option P&L; Decomposition

PURPOSE

Attribute short-horizon option price changes to spot, curvature, time decay, and volatility so model predictions can later be connected to trading P&L;

Core concepts

- Delta P&L;
- Gamma convexity
- Theta decay
- Vega exposure
- Taylor approximation and residual

Implementation checklist

- Compute start and end option values
- Freeze Greeks at the start for attribution
- Calculate each contribution
- Add a residual term
- Support scenario batches
- State scaling for dt and dSigma

Key equations

$$dC \approx \Delta * dS + 0.5 * \Gamma * (dS)^2 + \Theta * dt + \text{Vega} * d\text{Sigma}$$

$$\text{residual} = \text{actual_dC} - \text{approximated_dC}$$

Required experiments

- [] Small versus large stock move
- [] Volatility jump with flat spot
- [] One day passing
- [] Near-expiry versus far-expiry
- [] Compare first-order and second-order approximations

Evidence to save

- P&L; waterfall table
- Scenario comparison plot
- Approximation-residual plot
- Written sign interpretation

Common failure modes

- Double-counting time or volatility changes
- Using end-of-period Greeks for a start-based approximation
- Ignoring cross-Greeks over large moves
- Treating the residual as random noise without diagnosis

PASS CRITERION

Given an option price move, identify the major likely drivers and explain when the Greek approximation should be trusted or expected to break down.

Implied Volatility by Numerical Inversion

PURPOSE

Reverse Black-Scholes: infer the volatility input that reproduces an observed option price, while learning when numerical inversion is unreliable.

Core concepts

- IV as model-implied parameter
- Price bounds and existence
- Bisection robustness
- Newton-Raphson speed
- Vega and conditioning

Implementation checklist

- Validate option-price bounds
- Implement bisection
- Implement Newton-Raphson
- Add iteration and tolerance limits
- Create fallbacks for low vega
- Return status flags with every solution

Key equations

Find sigma such that $BS_price(\sigma) = market_price$
 $\sigma[n+1] = \sigma[n] - (BS_price(\sigma[n]) - market_price) / Vega(\sigma[n])$

Required experiments

- [] Recover a known generating sigma
- [] Test deep ITM and deep OTM options
- [] Test very short maturity
- [] Use bad initial guesses
- [] Compare iteration counts and failure cases

Evidence to save

- Two IV solvers
- Recovery-error table
- Convergence traces
- Failure-reason taxonomy

Common failure modes

- Solving impossible prices
- Returning a number without a convergence flag
- Newton steps leaving valid volatility bounds
- Ignoring dividend yield or option exercise style in real data

PASS CRITERION

Explain why IV is not directly observed and is not automatically a physical volatility forecast; it is the parameter that makes a selected pricing model match a market price.

Volatility Smile and Implied-Volatility Surface

PURPOSE

Move from individual IVs to structured cross-sections over strike and maturity, then make the geometric representation used by image-based ML explicit.

Core concepts

- Smile, smirk, skew, and term structure
- Strike versus moneyness
- Log-moneyness and delta coordinates
- Irregular quotes versus regular grids
- Interpolation as a modeling choice

Implementation checklist

- Create a tidy option-IV table
- Plot IV by strike for each maturity
- Convert to one moneyness convention
- Define a fixed maturity-moneyness grid
- Interpolate with masks
- Export a matrix and image

Key equations

```

moneyness = K / S
log_moneyness = log(K / S)
IV_surface = sigma_IV(moneyness, maturity)
grid X[t] in R^(H x W), for example 10 x 18

```

Required experiments

- [] Create flat, smile, skew, and term-structure examples
- [] Compare interpolation methods
- [] Visualize observed versus interpolated cells
- [] Test scale and color-map consistency across dates

Evidence to save

- Smile curves
- Heatmap and 3D surface
- Grid specification
- Missingness mask

Common failure modes

- Treating strike grids as comparable when spot changes
- Interpolating across unsupported regions
- Using per-image color scaling that leaks relative levels
- Discarding the missingness pattern

PASS CRITERION

Look at an IV image and explain both axes, every cell value, which values were observed versus interpolated, and what economic pattern the geometry represents.

THESIS BRIDGE

This is the direct bridge to the Kelly et al. setup: an IV surface becomes a structured input, not an arbitrary picture. Your axes, grid, masks, and normalization must be economically defensible.

Physical P Versus Risk-Neutral Q

PURPOSE

Separate the world used for forecasting realized outcomes from the measure used to price replicable claims. This prevents conceptual errors when ML uses both historical and option-implied variables.

Core concepts

- Physical dynamics
- Risk-neutral dynamics
- Drift replacement
- Discounted expectation pricing
- Pricing information versus real-world prediction

Implementation checklist

- Simulate terminal prices under P
- Simulate terminal prices under Q
- Overlay terminal distributions
- Estimate both expectations
- Price a call under Q
- Compare with Black-Scholes

Key equations

Under P: $dS = \mu S dt + \sigma S dW_P$
 Under Q: $dS = r S dt + \sigma S dW_Q$
 $E_P[S_T] = S_0 \exp(\mu T)$
 $E_Q[S_T] = S_0 \exp(r T)$
 $C_0 = \exp(-r T) * E_Q[\max(S_T - K, 0)]$

Required experiments

- [] Change μ while holding pricing inputs fixed
- [] Change r and observe Q distribution and price
- [] Show why pricing under P generally fails
- [] Explain what an IV input contributes to a predictive model

Evidence to save

- P/Q distribution plot
- Expectation-check table
- Monte Carlo call comparison
- Forecasting-versus-pricing note

Common failure modes

- Calling Q the real probability
- Using μ in a no-arbitrage pricer
- Claiming P and Q variables must be transformed to the same measure before they can coexist as features
- Ignoring the prediction target's measure

PASS CRITERION

Explain why an ML model trained on historical data does not become risk-neutral merely because its input includes implied volatility, and why IV may still contain predictive information.

Monte Carlo Option Pricing

PURPOSE

Build a generic risk-neutral simulation pricer, quantify sampling error, and distinguish pricing simulation from synthetic-data generation.

Core concepts

- Discounted payoff expectation
- Estimator variance
- Standard error
- Convergence rate
- Antithetic sampling

Implementation checklist

- Simulate terminal S under Q
- Discount average payoffs
- Compute standard errors and confidence intervals
- Measure runtime
- Implement antithetic variates
- Compare with the BS benchmark

Key equations

```
C0_hat = exp(-r*T) * (1/N) * sum(max(S_T[i]-K, 0))
standard_error = sample_std(discounted_payoffs) / sqrt(N)
sampling_error is proportional to 1/sqrt(N)
```

Required experiments

- [] Use $N = 10^2$ through 10^6
- [] Repeat each N across random seeds
- [] Plot absolute error and standard error
- [] Compare plain and antithetic estimators
- [] Check empirical confidence-interval coverage

Evidence to save

- MC pricer
- Convergence plot
- Runtime table
- Variance-reduction comparison

Common failure modes

- One lucky run presented as convergence
- No uncertainty estimate
- Simulating under P for risk-neutral pricing
- Antithetic implementation that accidentally doubles rather than pairs samples

PASS CRITERION

Explain when simulation is used to price an option and when it is merely used to create synthetic observations; state why increasing N has diminishing returns.

Heston Simulation and an Emergent Smile

PURPOSE

Learn why stochastic volatility exists by simulating it, pricing options, and converting those prices back into Black-Scholes IVs. Avoid beginning with the analytic Heston formula.

Core concepts

- Stochastic variance
- Mean reversion
- Volatility of volatility
- Spot-vol correlation
- Model-generated smile

Implementation checklist

- Generate correlated normals
- Use a non-negative variance scheme
- Simulate spot and variance paths
- Price an option grid under Q
- Invert prices to BS IV
- Package parameters in a configuration object

Key equations

```
dS = mu*S*dt + sqrt(v)*S*dW_S
dv = kappa*(theta-v)*dt + xi*sqrt(v)*dW_v
dW_S*dW_v = rho*dt
Feller guide: 2*kappa*theta >= xi^2
```

Required experiments

- [] Vary kappa, theta, xi, and rho one at a time
- [] Compare rho < 0, 0, and > 0
- [] Inspect discretization bias
- [] Generate a strike-maturity IV surface
- [] Compare with constant-volatility GBM

Evidence to save

- Spot/variance path plots
- Parameter-sensitivity panels
- Synthetic option table
- Recovered BS-IV surface

Common failure modes

- Negative Euler variance without correction
- Using mu instead of r for pricing
- Changing several parameters at once
- Interpreting one simulated surface as a calibrated market model

PASS CRITERION

Explain why Heston exists and trace the full chain: stochastic variance -> option prices -> Black-Scholes implied volatilities -> a non-flat surface.

Option-Surface Arbitrage Sanity Checks

PURPOSE

Understand why raw quotes are not a clean IV surface. Detect obvious static inconsistencies before interpolation or model training.

Core concepts

- Monotonicity in strike
- Convexity in strike
- Price bounds
- Calendar consistency
- Quote noise and IV instability

Implementation checklist

- Generate a valid synthetic chain
- Inject duplicates and bad quotes
- Check bounds
- Check discrete strike monotonicity
- Check discrete convexity
- Attach flags instead of silently deleting

Key equations

```
partial C / partial K <= 0
partial^2 C / partial K^2 >= 0
max(S-K*exp(-r*T),0) <= C <= S
```

Required experiments

- [] Map one bad quote into its IV distortion
- [] Test tolerances under bid-ask noise
- [] Compare checks before and after interpolation
- [] Visualize violations on the surface grid

Evidence to save

- Validation functions
- Flagged quote table
- Violation heatmap
- Cleaning-policy log

Common failure modes

- Applying continuous inequalities without numerical tolerance
- Silently clipping all violations
- Checking IV monotonicity where price monotonicity is the relevant condition
- Assuming every calendar pattern is trivial with rates and dividends

PASS CRITERION

Explain why an apparently extreme IV may be a quote or data-pipeline problem, and describe which checks should occur before interpolation and ML normalization.

THESIS BRIDGE

Bad quotes, inconsistent filters, and interpolation artifacts can become learnable image features. A cleaning flag and missingness mask are part of the research data, not housekeeping details.

From an Option Chain to an ML Tensor

PURPOSE

Build the complete data-engineering bridge from raw-ish option rows to the exact tensor consumed by a neural network, with leakage controls and auditable transformations.

Core concepts

- Tidy option-chain schema
- IV inversion
- Coordinate choice
- Regular grids
- Date alignment, normalization, and labels

Implementation checklist

- Parse dates, expiries, option type, and spot
- Remove or flag duplicates
- Compute time to maturity
- Calculate or validate IV
- Map onto a fixed grid
- Create masks and normalization statistics
- Align features and future labels
- Save tensors plus metadata

Key equations

option_price -> IV -> moneyness -> maturity -> regular_grid -> tensor
 $X[t]$ in $\mathbb{R}^{(10 \times 18)}$ (example only)
 label $y[t+h]$ must use information strictly after feature time t

Required experiments

- [] Visualize several dates with a fixed scale
- [] Compare train-only versus full-sample normalization
- [] Audit edge dates for leakage
- [] Measure grid coverage
- [] Reconstruct a human-readable surface from a saved tensor

Evidence to save

- Data dictionary
- Transformation pipeline
- Tensor, masks, and metadata
- Leakage audit
- Grid-coverage report

Common failure modes

- Randomly splitting dates
- Fitting normalization on validation or test data
- Losing identifiers during aggregation
- Treating interpolated values as observed
- Constructing labels with overlapping future information

PASS CRITERION

Select one final tensor cell and trace it backward to its source quote, date, maturity, coordinate transformation, interpolation decision, and normalization value.

THESIS BRIDGE

This is the most important notebook for your IV-surface thesis. Model sophistication cannot repair an unauditable tensor, unstable coordinates, or target leakage.

Tiny End-to-End Volatility Research Project

PURPOSE

Combine finance, data engineering, and correct ML methodology in one deliberately simple study before introducing CNNs, ViTs, text transformers, or multimodal fusion.

Core concepts

- Research question and target
- Chronological validation
- Baselines
- Forecast metrics
- Economic interpretation and uncertainty

Implementation checklist

- State one falsifiable question
- Build naive persistence and historical-mean baselines
- Fit linear regression
- Fit ridge regression
- Optionally fit a small MLP
- Tune on validation only
- Evaluate once on the held-out test period
- Document all feature timestamps

Key equations

Example A: $IV_surface[t] \rightarrow \text{predicted } RV[t+1]$
 Example B: $IV_surface[t] \rightarrow \text{predicted } IV_surface[t+1]$
 train dates < validation dates < test dates

Required experiments

- [] Compare every model against naive baselines
- [] Run an ablation without surface geometry
- [] Test at least two evaluation windows
- [] Report prediction error by market regime or maturity
- [] Inspect whether improvement survives a simple cost or robustness check

Evidence to save

- One-page research design
- Chronological split diagram
- Baseline/model results table
- Error analysis
- Reproducible final pipeline

Common failure modes

- Starting with a Transformer
- Random train-test split
- Weak or missing baselines
- Hyperparameter tuning on the test period
- Confusing predictive accuracy with a profitable strategy

PASS CRITERION

Defend the target, information set, split, baseline, metric, and interpretation. Explain what the model predicts, what it does not prove, and what evidence would justify moving to a more complex architecture.

THESIS BRIDGE

Only after this notebook should the thesis architecture progress from linear/ridge baselines to CNN, then ViT, then text Transformer, and finally multimodal fusion.

The chain you must be able to explain

Completion is conceptual, not ceremonial. Take a blank sheet and reconstruct this chain without notes:

```
observed stock prices S[t]
-> real-world dynamics under P:  $dS = \mu * S * dt + \sigma * S * dW_P$ 
-> risk-neutral dynamics under Q:  $dS = r * S * dt + \sigma * S * dW_Q$ 
-> pricing:  $C[t] = \text{discounted } E_Q[\text{payoff}]$ 
-> Black-Scholes price  $BS(S, K, T, r, \sigma)$ 
-> observed market option price  $C_{\text{market}}$ 
-> invert BS:  $\sigma_{IV}(K, T)$ 
-> regular IV grid  $X[t]$  in  $R^{(H \times W)}$ 
-> model  $f_{\theta}(X[t])$ 
-> future target  $y_{\text{hat}}[t+1]$ 
```

Oral defense prompts

- What is observed, what is estimated, what is simulated, and what is implied?
- Which objects live under P, which are used for Q pricing, and why may both appear in an ML dataset?
- Why does the Black-Scholes formula not require estimating the stock's physical drift?
- How do market price, implied volatility, and realized volatility differ?
- What creates discrete delta-hedging error?
- How can quote errors become extreme IV values or visual artifacts?
- How does one raw option quote become one tensor cell?
- Where could information from the future enter your pipeline?
- Why must a complex model beat persistence, linear, and ridge baselines?

Go / repair rubric

Area	Go	Repair before ML escalation
Pricing	Parity, tree, BS, and MC agree within expected tolerance	Reliance on formulas without replication or bounds intuition
Risk	Greeks and hedge errors have correct signs, units, and magnitudes	P&L is not portable across financing remain unclear
IV	Solver failures are detected; smile/surface coordinates are traceable	Misinterpretation of an observed physical forecast
Measures	P and Q roles are distinct and correctly connected	You believe mixed-measure features require automatic transformation
Data	Every tensor cell is traceable and leakage checks are implemented	Interpolation, scaling, or dates are opaque
Research	Chronological baselines and held-out evaluation are credible	Architecture complexity substitutes for experimental design

STOP RULE

If all six areas are in the Go column and the chain above feels ordinary rather than mysterious, stop general derivatives study. Your likely bottleneck is now statistics, ML/DL, implementation, and research methodology.

Compact notation and project handoff

Notation

Symbol	Meaning
S, S_0	Underlying spot price
K	Strike price
T, dt	Time to maturity; time increment in years
r	Continuously compounded risk-free rate
μ	Physical expected return or drift
σ	Volatility parameter
C, P	European call and put values
P, Q	Physical and risk-neutral probability measures
Delta, Gamma	First and second spot sensitivities
Vega, Theta, Rho	Volatility, time, and rate sensitivities
IV, RV	Implied volatility and realized volatility
$X[t]$	ML feature tensor built using information available at time t

Recommended repository structure

```
derivatives-foundation/
notebooks/01_gbm.ipynb ... 15_end_to_end.ipynb
src/pricing.py src/greeks.py src/simulation.py
src/iv.py src/surfaces.py src/validation.py
tests/ figures/ data/README.md environment.yml
README.md completion_checklist.md
```

Research anchor

Kelly, Bryan; Kuznetsov, Boris; Malamud, Semyon; and Xu, Teng Andrea. Deep Learning from Implied Volatility Surfaces. 2026 version supplied with this project. Use it as motivation for the structured IV-surface representation and later CNN replication, not as a substitute for the pricing and data-pipeline foundations in this syllabus.

NEXT TRACK

After completion: strengthen time-series validation, PyTorch implementation, CNNs, vision transformers, text transformers, multimodal fusion, ablations, and research reproducibility - while adding finance topics only when the thesis creates a concrete dependency.